

# JC4003 Natural Language Processing

## Lecture 10: Transformer and Model Training for NLP Tasks

Dr Xiao Li      [xiao.li@abdn.ac.uk](mailto:xiao.li@abdn.ac.uk)

Dr Ruizhe Li      [ruizhe.li@abdn.ac.uk](mailto:ruizhe.li@abdn.ac.uk)

Dr Siwei Liu      [siwei.liu@abdn.ac.uk](mailto:siwei.liu@abdn.ac.uk)

# Outline

Introduction to neural network

Recurrent Neural Networks (RNN, LSTM)

RNNs for NLP Tasks

Attention mechanism

Transformer

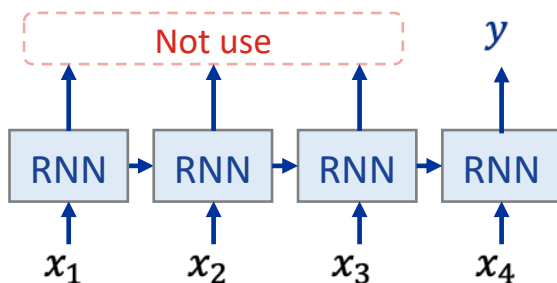
Model Training for NLP Tasks

# Attention Mechanism

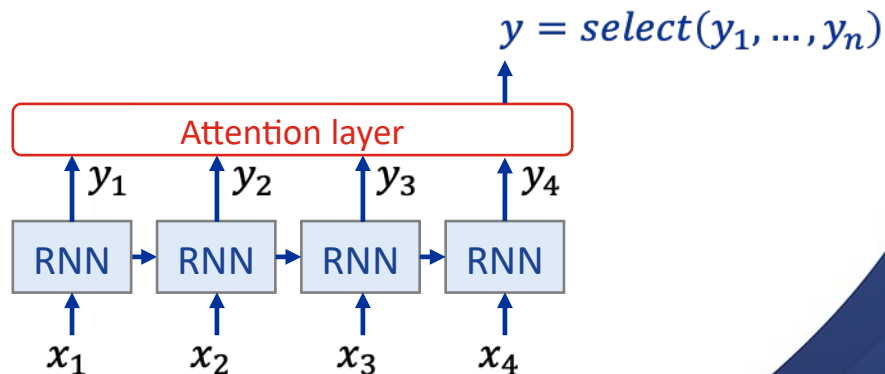
# Motivation

Initially, the Attention mechanism was designed to improve traditional RNN models and to overcome the difficulty of **capturing long-range dependencies**.

When processing long sequences, an attention layer provides a mechanism dynamically **selecting and focusing on the most relevant parts of the input** sequence to enhance the model's performance.



A traditional RNN model



RNN model with attention

# Attention Mechanism

The **initial design** of attention involves **many inputs** ( $y_1, \dots, y_n$ ) and one output ( $\hat{y}$ ).

It can be regarded as a process of **combining multiple potential outputs into one** by using a **weighted average**. The weights used in this process represent the degree of focus or attention given to each potential output.

$$\hat{y} = a_1 y_1 + \dots + a_n y_n$$

$$\Leftrightarrow \hat{y} = [a_1, \dots, a_n] \begin{bmatrix} y_1 \\ \vdots \\ y_n \end{bmatrix}$$

$$\Leftrightarrow \hat{y} = AY$$

# Attention Mechanism (cont.)

However, it doesn't matter if we extend the attention mechanism to have **multiple outputs**  $(h_1, \dots, h_n)$  by adopting more weights  $(a_{1,1}, a_{1,2}, \dots, a_{n,n})$ , i.e.:

$$\begin{aligned} h_1 &= a_{1,1}y_1 + \dots + a_{1,n}y_n \\ h_2 &= a_{2,1}y_1 + \dots + a_{2,n}y_n \\ &\dots \\ h_n &= a_{n,1}y_1 + \dots + a_{n,n}y_n \end{aligned} \Leftrightarrow \begin{bmatrix} h_1 \\ \vdots \\ h_n \end{bmatrix} = \begin{bmatrix} a_{1,1} & \dots & a_{1,n} \\ \vdots & \ddots & \vdots \\ a_{n,1} & \dots & a_{n,n} \end{bmatrix} \begin{bmatrix} y_1 \\ \vdots \\ y_n \end{bmatrix} \Leftrightarrow H = AY$$

This is widely used in deep learning models when calculating intermediate results.

# Types of Attention Layers

There are various types of attention layers, and the Attention mechanism can be categorised based on different criteria.

## Based on the directionality of attention

- Self-Attention
- Cross-Attention

## Based on how attention weights are calculated

- Additive Attention
- Dot Product Attention
- Scaled Dot Product Attention

# Self-Attention & Cross-Attention

- **Self-Attention** (also known as Intra-Attention), this type of attention calculates the attention weights (**A**) within different parts of the same input sequence (**Y**). It helps in capturing dependencies between elements of the input sequence.

$$Attention(Y) = AY = f(Y)Y$$

- **Cross-Attention** calculates the relationships between two different sequences (**Y** and **Z**), i.e. the calculation of attention weights (**A**) requires the additional input (**Z**).

$$Attention(Y, Z) = AY = f(Y, Z)Y$$

NB: **Self-Attention** can be also regarded as **Cross-Attention** where  $Z = Y$ .

# Additive Attention

Additive attention is the **earliest form** of the attention mechanism. It simply joins all the inputs as a long vector, and uses a feedforward neural network to calculate attention weights.

$$\text{AdditiveAttn}(Y, Z) = AY = W(Y \oplus Z)Y$$

where  $\oplus$  denotes the concatenation of  $Y$  and  $Z$ , and  $W$  is a learnable matrix.

# Dot Product Attention

Dot product attention has 3 input parameters, query (**Q**), key (**K**), and value (**V**). Attention weights (**A**) are obtained by calculating the dot product between query (**Q**) and key (**K**), and the attention weights are then normalised using a **Softmax** function to produce attention weights. The output is then obtained by applying **A** on value (**V**).

$$\text{DotProductAttn}(Q, K, V) = AV = \text{softmax}(QK^T)V$$

- For **Self-Attention**,  $Q = K = V = Y$ , i.e.:

$$\text{SelfAttn}(Y) = \text{DotProductAttn}(Y, Y, Y)$$

- For **Cross-Attention**,  $Q = Y, K = V = Z$

$$\text{CrossAttn}(Y, Z) = \text{DotProductAttn}(Y, Z, Z)$$

# Scaled Dot Product Attention

Scaled Dot-Product Attention differs from Dot-Product Attention by **scaling the dot product of the query and key by the square root of the key dimension ( $d_k$ )**.

NB: usually dimensions of query and key are the same, i.e.  $d_k = d_q$ , so it **doesn't matter whether we use  $d_k$  or  $d_q$** .

$$\text{ScaledDotProductAttn}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \mathbf{AV} = \text{softmax}\left(\frac{\mathbf{QK}^T}{\sqrt{d_k}}\right) \mathbf{V}$$

This helps to **mitigate the issue of extremely large values** in the softmax function when dealing with high-dimensional keys. The extremely large values will cause the gradient of the softmax function to become very small, making it difficult for the model to learn during training.

# Transformer

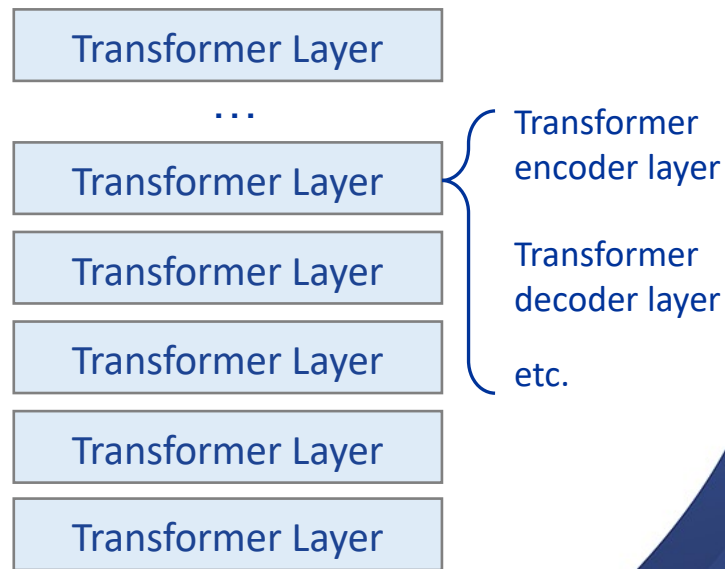
# Transformer

The Transformer is a **groundbreaking** model architecture in deep learning, particularly in the field of NLP.

Unlike traditional RNNs the **Transformer is entirely based on the Attention mechanism**, allowing it to process sequences in **parallel** rather than sequentially.

The architecture of a Transformer is based on **stacking multiple Transformer layers**.

Depending on the implementation, it might use encoder layers, decoder layers, or a combination of both.



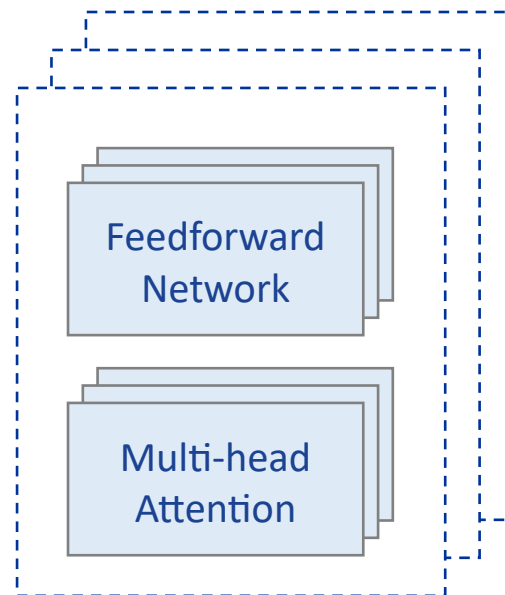
# Transformer Layers

There are many Transformer layers for different takes:

- Transformer Encoder layer
- Transformer Decoder layer
- or their variants

Regardless of the type of the layers, each layer is composed of 3 types of components:

- multi-head attention
- feedforward network (FFN)
- layer normalisation



Types of Transformer Layer

# Multi-head Attention

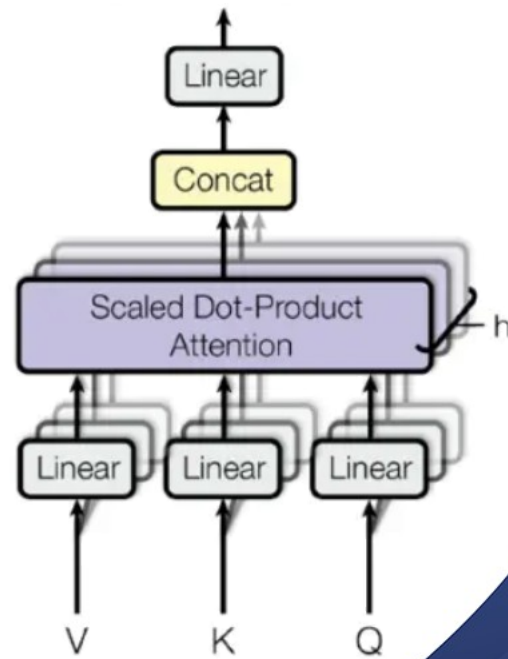
Multi-head attention **extends Scaled Dot-Product Attention** by using multiple attention heads to focus on different parts of the input simultaneously, allowing the model to capture a wider range of relationships within the data.

A Multi-head Attention is **composed of several** Scaled Dot-Product Attention mechanisms running in parallel.

$$MultiHead(Q, K, V) = W(H_1 \oplus H_2 \oplus \dots \oplus H_h)$$

$$H_i = ScaledDotProduct(W_{i,q}Q, W_{i,k}K, W_{i,v}V)$$

where  $W, W_{i,q}, W_{i,k}, W_{i,v}$  are weight matrixes,  $\oplus$  denotes the concatenation,  $h$  denotes the number of heads.



Multi-head  
Attention

# Feedforward Network (FFN)

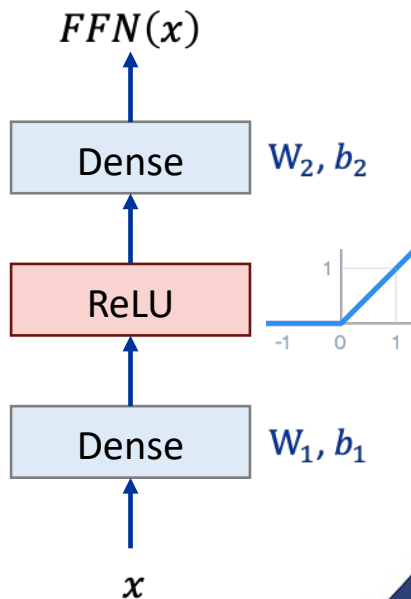
A Feedforward Network in the Transformer is a **two-layer fully connected network** with a **ReLU** activation in between.

The first layer's output dimension is doubled before being reduced back to the original size in the second layer.

$$FFN(x) = W_2 Relu(W_1 x + b_1) + b_2$$

$$Relu(x) = \max(0, x)$$

where  $W_2, W_1, b_1, b_2$  are the learnable parameters.

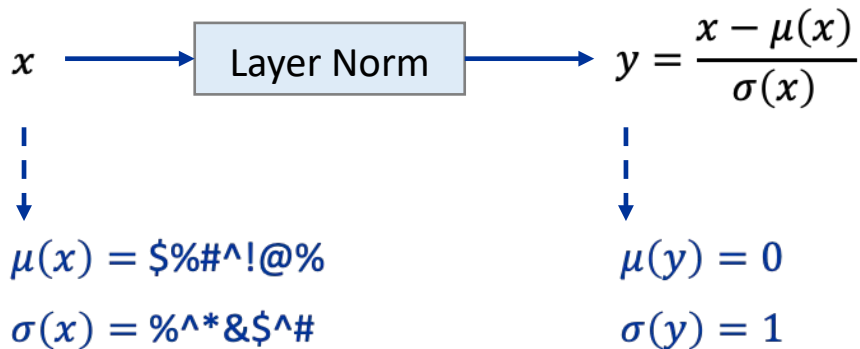


Feedforward  
Network

# Layer Normalisation

Layer normalisation is **one of many forms of normalisation** that can be used to **improve training performance** in deep neural networks by keeping the values of a hidden layer in a range that facilitates gradient-based training.

**For each input tensor (in a batch)**, layer normalisation normalises it by **adjusting its mean and variance**, and then outputs a transformed vector with the same dimensionality, but its mean and variance are adjusted to **0 and 1** respectively.



# Transformer Encoder Layer

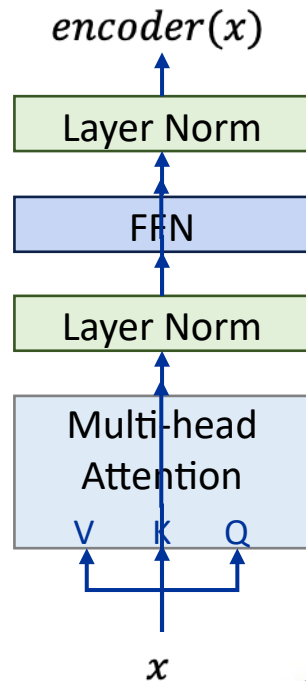
A Transformer Encoder Layer consists of a **multi-head self-attention** followed by a **FFN**, with each sub-layer surrounded by **layer normalisation** and **residual connections**.

$$H_1 = \text{MultiHead}(x, x, x) + x$$

$$H_2 = \text{Norm}(H_1)$$

$$H_3 = \text{FFN}(H_2) + H_2$$

$$\text{Encoder}(x) = \text{Norm}(H_3)$$



# Transformer Decoder Layer

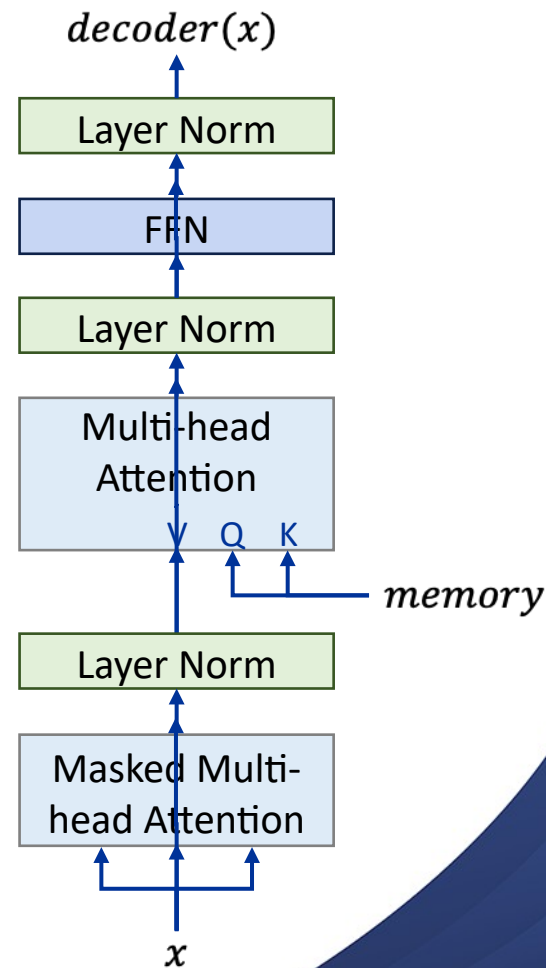
A Transformer Decoder Layer is similar to an Encoder Layer but with a **masked multi-head self-attention**.

After the masked self-attention, it also includes a standard multi-head attention mechanism, where the queries come from the decoder and the keys and values come from the encoder's output (called **memory**), then an FFN.

$$H_1 = \text{Norm}(\text{MaskedMultiHead}(x, x, x) + x)$$

$$H_2 = \text{Norm}(\text{MultiHead}(m, m, x) + x)$$

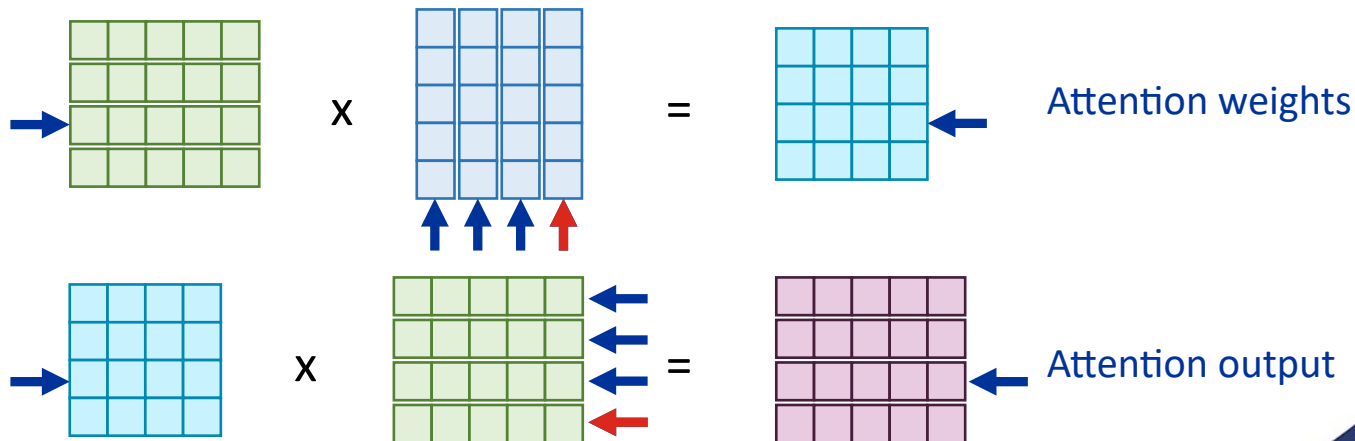
$$\text{Encoder}(x) = \text{Norm}(\text{FFN}(H_2) + H_2)$$



# Masked Attention

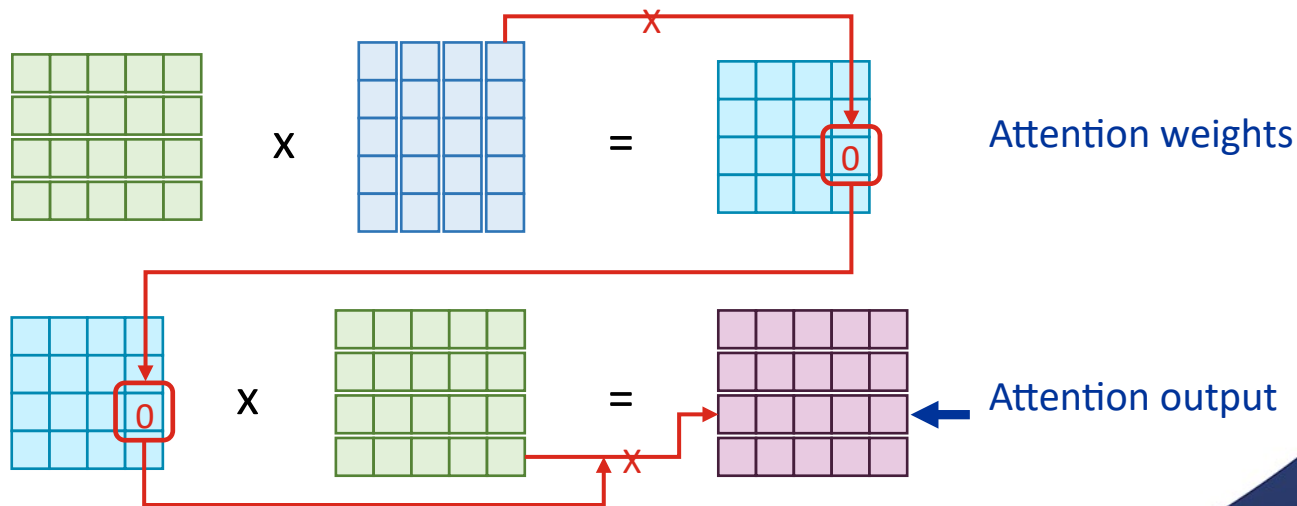
Typically, attention weights are calculated based on all elements of the input, and each output of the attention mechanism is a weighted average of all the input elements.

However, this becomes problematic in certain NLP tasks where the processing of a particular word should not depend on words that appear later in the sequence.



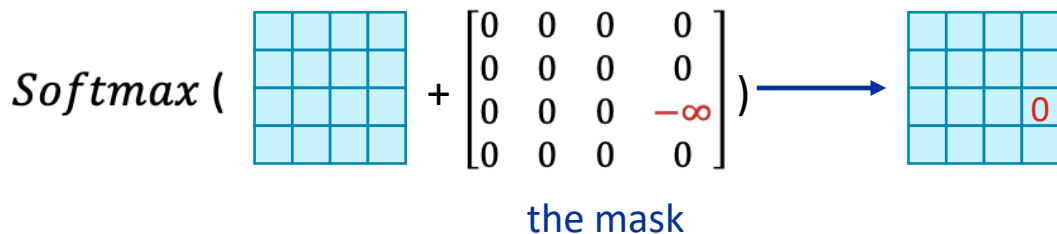
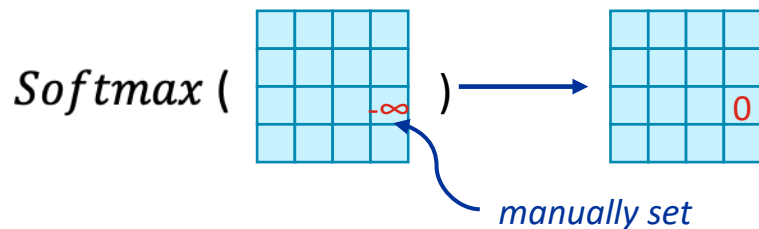
# Masked Attention (cont.)

We can prevent specific outputs from depending on certain inputs by manually setting some of the attention weight elements to 0.



# Masked Attention (cont.)

Since the softmax function used in the calculation of attention weights operates on logarithmic values, we can set certain elements to negative infinity before applying softmax, which will result in them becoming zero after the softmax computation.



# Masked Attention (cont.)

Thus, the Masked Scaled Dot Product Attention used in Transformer can be defined as follow:

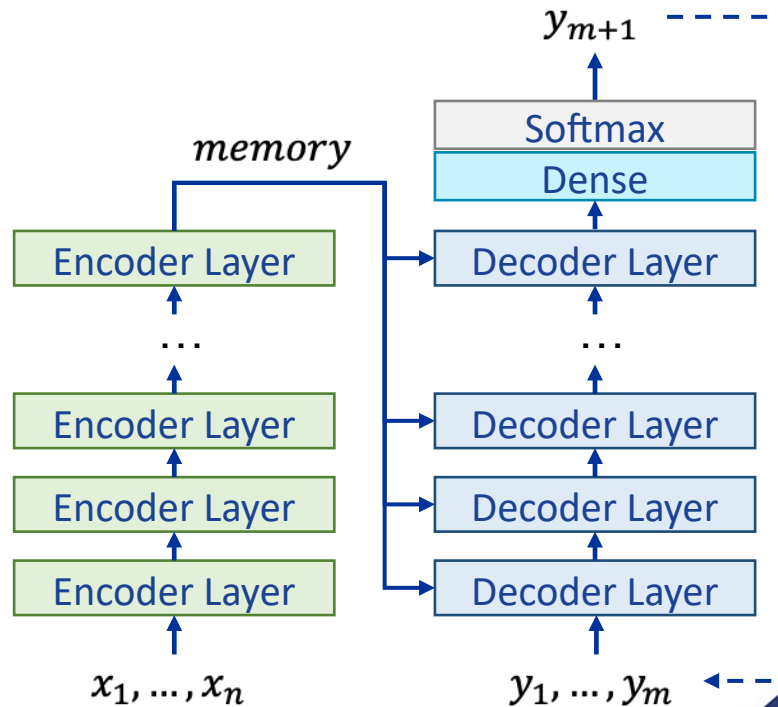
$$\textit{MaskedScaledDotProductAttn}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \mathbf{AV} = \text{softmax}\left(\frac{\mathbf{QK}^T}{\sqrt{d_k}} + \mathbf{M}\right)\mathbf{V}$$

where  $\mathbf{M}$  denotes the mask consisting of 0 and  $-\infty$ . When entries are  $-\infty$ , the related dependences will be **blocked**.

# Transformer Encoder-Decoder Architecture

The complete Transformer architecture is an **Encoder-Decoder** model designed for **sequence-to-sequence tasks**, such as machine translation, where it transforms an input sequence into an output sequence.

- encoder outputs the *memory*
- each decoder layer receives *memory*
- the previously output tokens are input back into the decoder, and
- the decoder only generates **one next token** at a time (called **greedy decoding**)



Transformer

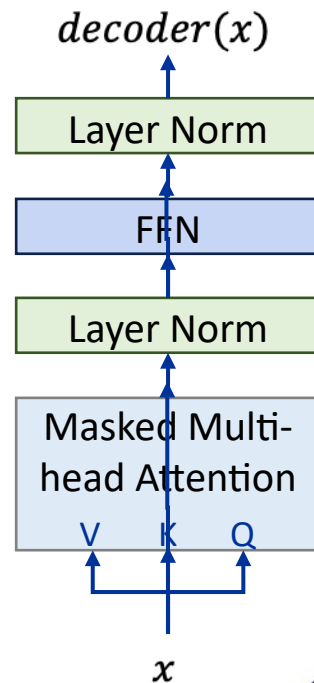
# Transformer Decoder Layer (GPT version)

The variant of the Transformer **decoder** used in GPT (Generative Pre-trained Transformer) differs from the standard Transformer decoder as it **excludes Encoder-Decoder Attention**.

This is because the Transformer architecture in GPT only consists of decoder layers, with no encoder. Thus, GPT's decoder exclusively relies on self-attention, as the model **generates text purely based on the previous tokens** in the sequence, without the need for a separate encoder.

$$H = \text{Norm}(\text{MaskedMultiHead}(x, x, x) + x)$$

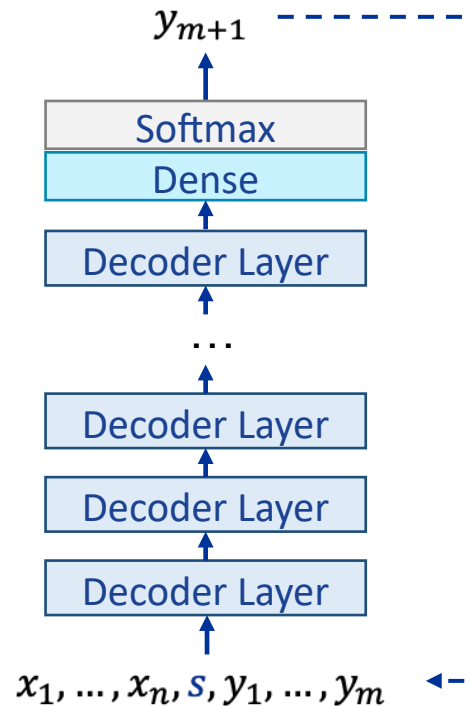
$$\text{Decoder}(x) = \text{Norm}(\text{FFN}(H) + H)$$



# Transformer Architecture for GPT

The GPT model **discards the encoder** and is **entirely composed of decoders**, which results in the removal of the attention layer that typically handles *memory*.

Since there is no encoder, GPT directly places the encoder's input ( $x_1, \dots, x_n$ ) in front of the decoder's input ( $y_1, \dots, y_m$ ), appending it before the decoder's output, separated by a special delimiter ( $s$ ).



Transformer for GPT

# Position Embedding

# Position Embedding

Transformers fully **replace the recurrent units with attention**. Unlike RNNs, the attention computation across tokens can be fully parallelised, that is, they do not have to wait for the calculation of the previous token's representation to get the current token's representation.

Consequently, **a natural limitation of the Transformer is that it lacks inherent positional information**.

Without positional encoding, the Transformer is permutation-invariant as an operation on sets (due to the commutative property of addition, in attention). For example, **“Alice follows Bob”** and **“Bob follows Alice”** are completely different sentences, but a Transformer without position information will produce the same representation.

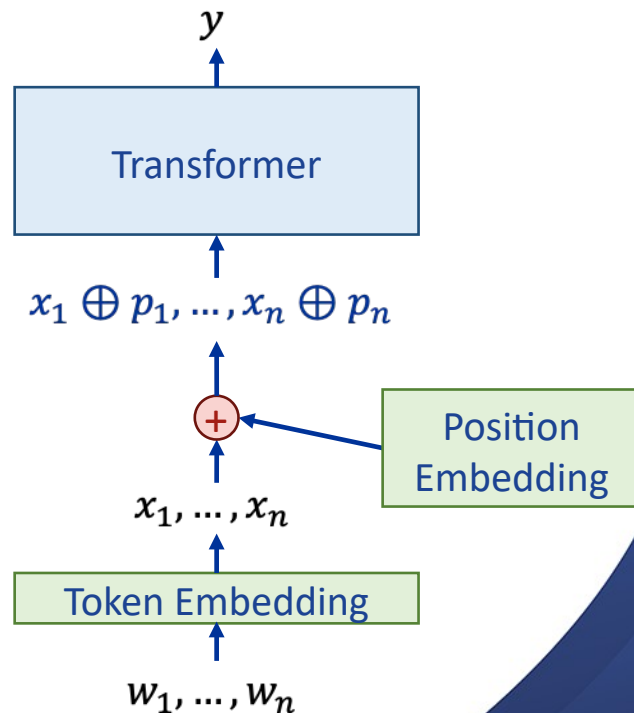
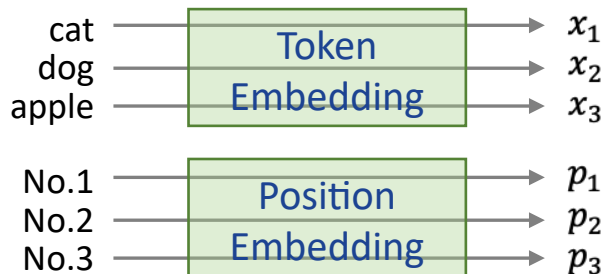
Therefore, the **Transformer needs to explicitly encode the position information**.

# Position Embedding (cont.)

In addition to token embeddings (i.e. word embeddings), Transformers require position embeddings to provide information about the position of each token in the sequence.

Position embeddings are additional vectors  $(p_1, \dots, p_n)$ , concatenated to each input vector  $(x_1, \dots, x_n)$  individually.

Just as word embeddings assign an independent vector to each word, position embeddings assign an independent vector to each position (or position index).



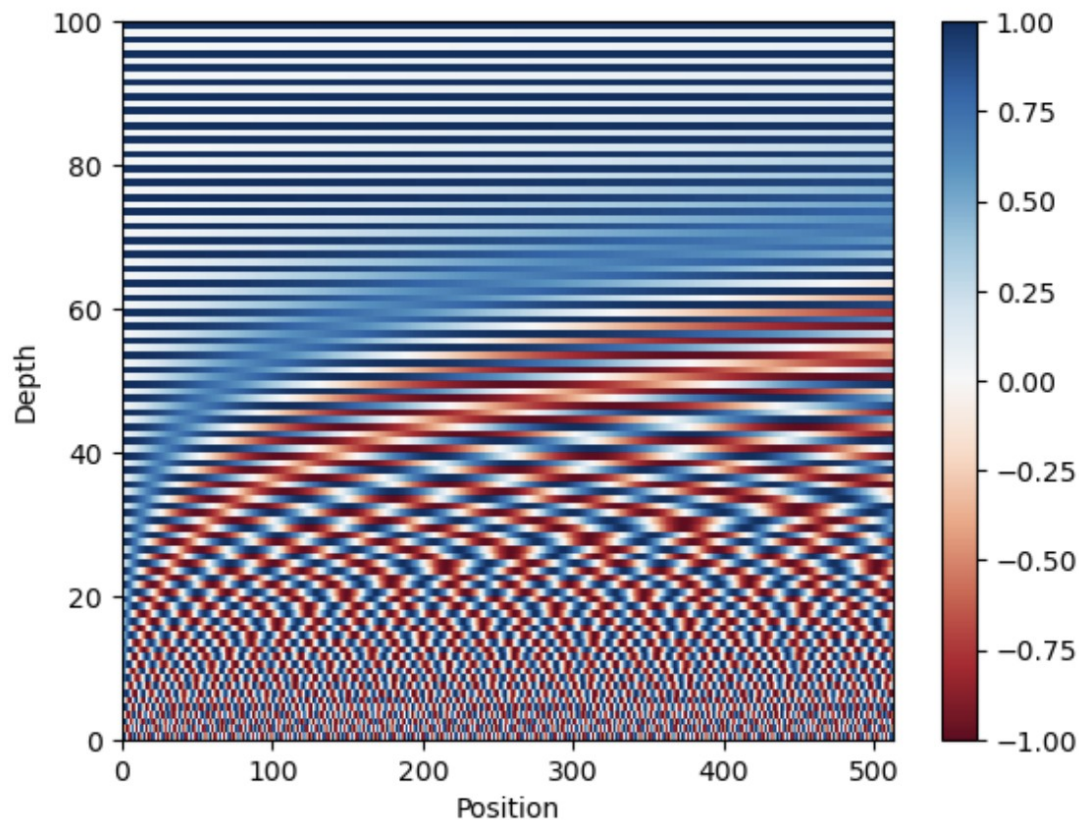
# Position Embedding (cont.)

There are two methods to obtain position embeddings:

**Learned Position Embeddings:** In this approach, each position in the input sequence is assigned a unique embedding vector, which is **learned during the training** process. Similar to how word embeddings are learned, the model treats the position as an additional input feature and adjusts the embeddings as it optimises its performance.

**Sinusoidal Position Embeddings:** This method involves using **fixed sinusoidal functions** to generate position embeddings. The embeddings are determined based on **sine and cosine functions of different frequencies**, which encode the positions in a way that preserves relative distances between tokens. This approach does not require learning the embeddings and ensures that the model can generalise to sequences of different lengths.

# Sinusoidal Position Embeddings



# The Training for NLP Models

# The Training for NLP Models

NLP model training varies depending on the model and the task.

For example, the training process will be different for [generating classifications](#) or [generating texts](#).

Different models, such as [RNNs](#) and [Transformers](#), have distinct architectures, which influence the training process.

# Generating Classifications

In NLP, classification includes many tasks, whatever sentence level classification or token level classification, for example:

- **Sentiment Analysis** -- Classifying text as expressing **positive**, **negative**, or **neutral** sentiment.
- **Text Classification** -- **Categorising documents** or text passages into specific topics or categories, such as news article classification.
- **Language Detection** -- **Identifying the language** of a piece of text based on its content.
- **Spam Detection** -- Classifying emails or messages as either **spam** or **legitimate**.
- **POS Tagging** -- Classifying each word in a sentence into its corresponding part of speech, such as **noun**, **verb**, **adjective**, **etc**.

# Generating Classifications (RNNs)

Due to the recurrent structure of RNNs, when training an RNN model, **tokens  $(x_1, \dots, x_n)$  must be fed into the model one by one**. This is because an RNN can only process one token at a time, relying on the passage of time to receive different tokens at each time step.

$$o_1, h_1 = RNN(x_1, h_0)$$

$$o_2, h_2 = RNN(x_2, h_1)$$

...

$$o_n, h_n = RNN(x_n, h_{n-1})$$

where  $o_1, \dots, o_n$  are the RNN outputs,  $h_0, \dots, h_n$  are the RNN hidden state (usually  $h_0$  is a zero vector).

NB: usually deep learning frameworks (e.g. PyTorch) provide utility functions that automatically handle the iteration of RNNs.



# Generating Classifications (loss)

When the outputs  $o_1, \dots, o_n$  are obtained, it doesn't matter whether you **use all of them** or **only the last one**. For classification tasks, the **cross-entropy loss** is typically used, i.e.:

$$\mathcal{L} = \text{CrossEntropy}(o_n, t) \quad \text{or} \quad \mathcal{L} = \sum_i \text{CrossEntropy}(o_i, t_i)$$

where  $t$  and  $t_i$  denotes the ground true labels.

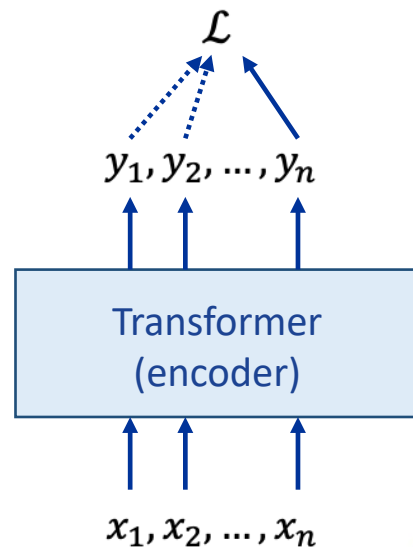
NB: In cross-entropy, the exponential operation is equivalent to a softmax function. If the model already includes a softmax layer at the end, applying softmax again during the loss calculation can lead to numerical instability / incorrect gradients, etc. In this case, **NLLoss should be used instead of cross-entropy**, which is a version of cross-entropy without the softmax operation.

$$\text{NLLoss}(\text{Softmax}(o_n), t) = \text{CrossEntropy}(o_n, t)$$

# Generating Classifications (Transformer)

The training process of a Transformer is similar to that of an RNN, but due to the Transformer's support for parallelism, **all input tokens can be fed into the model simultaneously**, and the model can **generate all outputs at once**.

In classification tasks, it is usually **unnecessary to mask** subsequent tokens for each word, so the task is often handled by the Transformer encoder alone, making masking unnecessary.



# Generative Models

The output of an NLG (Natural Language Generation) model, also known as a generative model, is text.

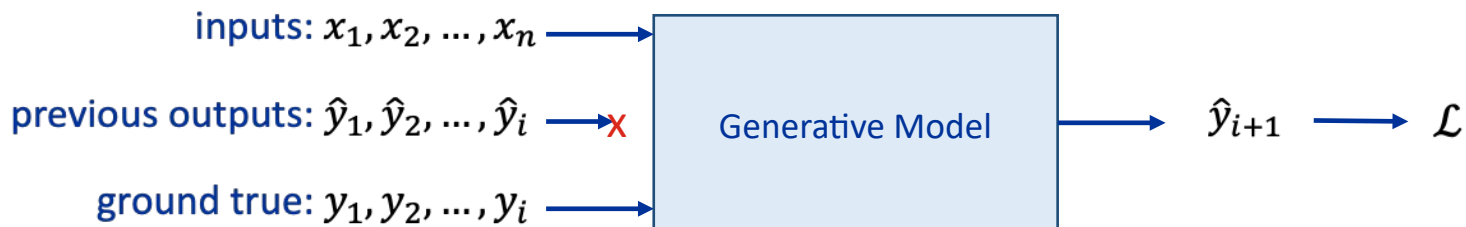
This differs from producing classification labels. When outputting labels, each label is only related to the corresponding token (or the entire sentence), whereas **generating text is not only related to the input but also to the outputs at different time steps.**

If a generative model is trained in the same way as a classification model, it will struggle to learn the contextual dependencies of the output text.

# Teaching Forcing

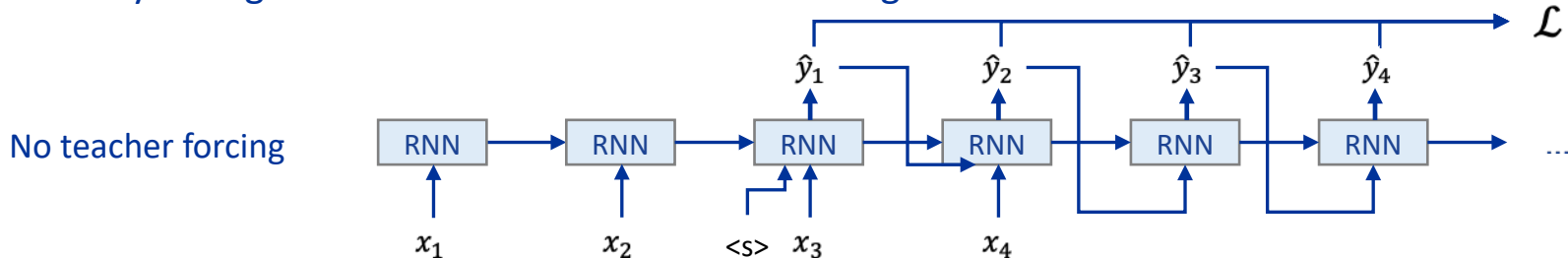
The training of generative models typically uses a method known as teacher forcing.

Teacher forcing involves providing the model with the **ground truth output from the previous time step** as the next input during training, **rather than using the model's own predictions**.

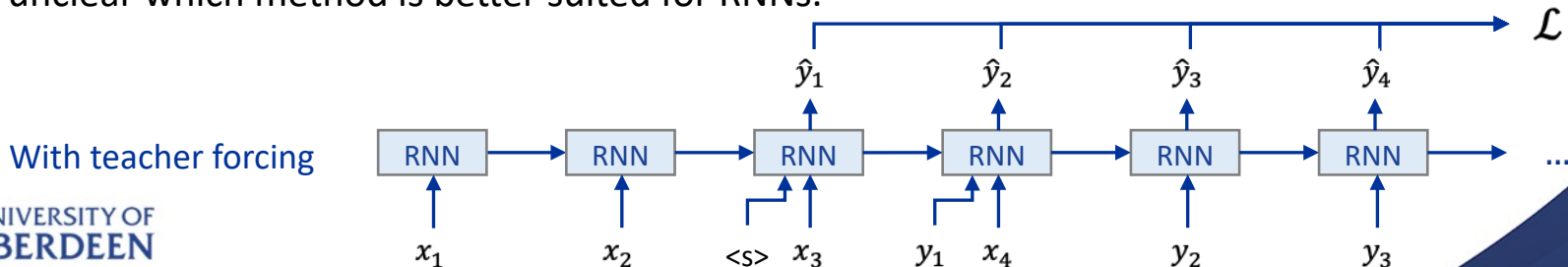


# Generative Models (RNN)

RNN models were used for text generation long before the introduction of teacher forcing, so many RNN generative models still use the training method of classification models.



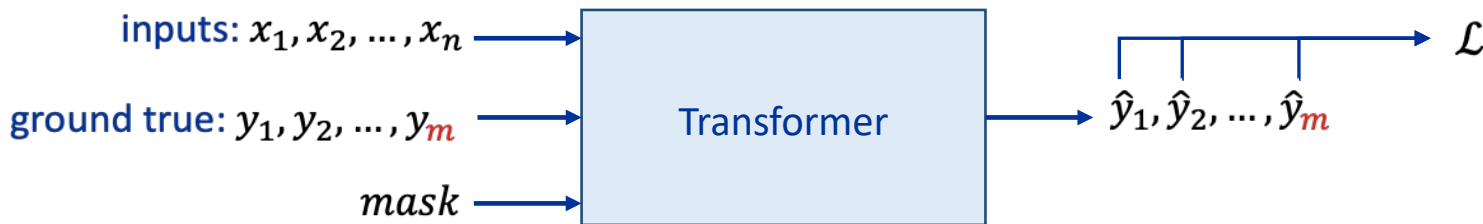
However, many RNNs also **use teacher forcing** or a **mix of both training methods**. It is unclear which method is better suited for RNNs.



# Generative Models (Transformer)

Due to the Transformer's support for parallelisation, it receives all inputs at once and outputs all results simultaneously, so **the training process must use teacher forcing**.

Additionally, to prevent the predicted words from seeing the correct subsequent tokens, **masking is required** to block the relevant inputs.



# Generative Models (Transformer)

Training Transformers is often much more challenging than training RNNs (although Transformers typically yield better results), and issues like numerical instability and overfitting can easily arise during the process. Therefore, when training Transformers, it's also important to pay attention to the following aspects:

- **Weight Initialisation:** Transformer models typically use standard deep learning initialisation methods, such as **Xavier initialisation** (also known as Glorot initialisation) or **He initialisation**.
- **Learning Rate Scheduling:** Transformer models usually employ special learning rate schedules, such as using a "**warm-up**" phase where a very low learning rate is initially used, followed by a gradual increase to a normal rate, and then a gradual decrease.
- **Regularisation:** To prevent overfitting, Transformer models generally require regularisation techniques, such as Dropout. Additionally, the scaled dot-product attention mechanism must be handled carefully to ensure numerical stability.

# Conclusion

**Attention Mechanism:** Introduced attention layers, enhancing models by focusing on relevant input parts and handling long-range dependencies.

**Transformer Model:** Covered how the Transformer processes sequences in parallel using multi-head attention and feedforward networks, replacing traditional RNNs.

**Training Models:** Discussed the training process for NLP models, including classification and generative tasks, using techniques like teacher forcing and masking.

**Position Embeddings:** Addressed the importance of adding position embeddings to encode token order in the sequence.

**Challenges in Transformer Training:** Highlighted key challenges, such as weight initialization, learning rate scheduling, and regularization techniques to prevent overfitting.